부산대학교
PUSAN NATIONAL UNIVERSITY

Arrays and Linked Lists

Same List, Different Costs

Choosing a Representation by Its Operations

DS3600082-140
자료구조
2026학년도 2학기

오용경
데이터사이언스전공
yongkyung.oh@pusan.ac.kr

Two List Representations

같은 리스트 연산도 내부 표현에 따라 다른 일을 한다

인터페이스의 연산

연산	입력	효과 또는 반환값	Python `list`
`get(i)`	인덱스 *i*	*i*번째 값 반환	`xs[i]`
`set(i,x)`	인덱스 *i*, 값 *x*	*i*번째 값을 *x*로 바꿈	`xs[i] = x`
`add(i,x)`	인덱스 *i*, 값 *x*	*x*를 *i*번째에 삽입	`xs.insert(i, x)`
`remove(i)`	인덱스 *i*	*i*번째 값을 삭제하고 반환	`xs.pop(i)`

Source: Morin (2013), Section 1.2.

ArrayStack의 내부 표현

ArrayStack은 배열(array)을 내부 배열(backing array)로 쓰고 n과 용량(capacity)을 따로 센다



인덱스 i가 주어지면 시작 주소에서 i칸 떨어진 위치를 바로 계산한다

Adapted from Morin (2013), Section 2.1.

SLList의 내부 표현

연결리스트(linked list)의 한 형태인 SLList는 노드마다 값과 `next`를 저장한다



> 노드의 메모리 위치와 목록에서의 순서는 다를 수 있다

Adapted from Morin (2013), Section 3.1.

DLList의 내부 표현

구성	노드가 저장하는 것	이동할 수 있는 방향
SLList	값, `next`	다음 노드
DLList	값, `prev`, `next`	이전 노드와 다음 노드

> 참조 하나를 더 저장하면 가까운 끝에서 출발할 수 있다

Source: Morin (2013), Section 3.2.

`get(3)`의 내부 실행 단계

같은 `get(3)` 호출도 내부 표현에 따라 서로 다른 단계로 실행된다

내부 표현	실행 단계
ArrayStack	시작 주소에서 세 칸 떨어진 위치 계산
SLList	`head`에서 `next`를 세 번 따라감
DLList	앞과 뒤 중 가까운 끝을 고른 뒤 따라감

Source: Morin (2013), Chapter 3 and Theorems 2.1, 3.1, 3.2.

호출하는 코드는 바뀌지 않는다

`xs`는 List 인터페이스를 따르는 객체이며 내부 표현이 달라도 같은 연산을 호출한다

```
def replace_third(xs, value):
    old = xs.get(2)
    xs.set(2, value)
    return old

```

배열 표현
위치를 계산한 뒤 `set`

연결리스트
노드를 따라간 뒤 `set`

> 같은 인터페이스 호출이 같은 내부 작업이나 같은 비용을 뜻하지는 않는다

연산 비용을 읽는 순서

연산 비용(operation cost)은 연산 하나에 필요한 시간이나 공간을 뜻한다

단계	질문	확인할 것
1	어떤 연산인가	`get`, `add`, `remove` 가운데 고름
2	내부에서 무엇을 하는가	주소 계산, 원소 이동, 노드 순회, 참조 변경
3	몇 번 반복하는가	n이나 i에 따라 반복 횟수를 셈

> `add(i,x)`는 배열에서 원소 이동, 연결리스트에서 노드 순회가 비용을 지배한다

Source: Morin (2013), Chapters 2 and 3.

Checkpoint

Two List Representations

- List 인터페이스가 내부 표현에 대해 말하지 않는 것은 무엇인가
- `get(i)`가 세 표현에서 각각 어떤 일을 하는가
- 같은 결과를 내면서도 연산 비용이 달라지는 까닭은 무엇인가

Section 2 • Arrays and Linked Lists

Two Access Patterns

위치에 닿는 방법이 접근 비용을 결정한다

배열은 위치를 계산한다

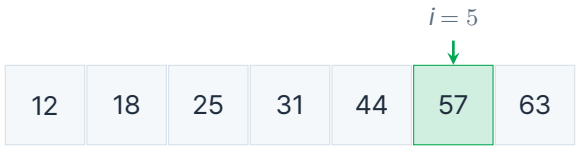
정의

임의 접근(random access)

배열의 i번째 원소 주소를 $\mathrm{addr}(a[i]) = \mathrm{base}(a) + i \times \mathrm{size}(\text{element})$ 로 계산해 한 번에 닿는 접근

Adapted from Morin (2013), Section 2.1.

배열에서 `get(i)`



> n과 i가 달라져도 `get(i)`는 $\Theta(1)$이다

Source: Morin (2013), Theorem 2.1.

SLList는 연결을 따라간다

순차 접근(sequential access)은 앞 노드부터 연결을 차례로 따라가는 방식이다

요청	따라간 횟수	접근 비용
`get(0)`	0	$\Theta(1)$
`get(3)`	3	노드 4개 확인
`get(i)`	i	$\Theta(i+1)$

Source: Morin (2013), Chapter 3.

DLList는 가까운 끝을 고른다



DLList의 접근 비용은 $\Theta(1 + \min\{i, n - i\})$이다

Source: Morin (2013), Theorem 3.2.

$n = 10$, $i = 6$에서 비교

표현	출발점	필요한 이동
ArrayStack	시작 주소	주소 계산 1회
SLList	`head`	`next` 6회
DLList	뒤쪽 센티널	`prev` 4회

Source: Morin (2013), Chapter 3 and Theorems 2.1, 3.1, 3.2.

처음부터 끝까지 훑는다면

순회는 표현이 달라도 모든 원소를 한 번씩 방문한다

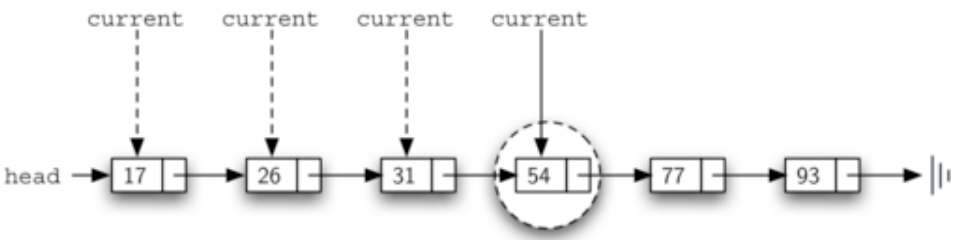
표현	다음 원소	n개 순회
배열	다음 인덱스	$\Theta(n)$
SLList	`next`	$\Theta(n)$
DLList	`next` 또는 `prev`	$\Theta(n)$

> 임의 접근의 차이는 전체 순회 비용을 바꾸지 않는다

Source: Morin (2013), Chapters 2 and 3.

정렬된 연결리스트에서 탐색

찾는 값보다 크면 일찍 멈추지만 가운데 노드로 바로 갈 수 없어 최악의 경우 $O(n)$이다



45를 찾는 동안 `current`가 17, 26, 31, 54를 차례로 가리키고 45보다 큰 54에서 멈춘다

Source: Miller and Ranum (2013), Implementing an Ordered List.

Checkpoint

Two Access Patterns

- 배열에서 `get(i)`가 $\Theta(1)$인 까닭은 무엇인가
- SLList와 DLList가 *i*번째 노드에 닿는 방법은 어떻게 다른가
- 세 표현의 전체 순회가 모두 $\Theta(n)$인 까닭은 무엇인가

Insert and Remove Costs

위치를 찾는 일과 구조를 바꾸는 일을 나누어 센다

삽입 비용을 찾기와 변경으로 나눈다

단계	배열	연결리스트
바꿀 위치 찾기	주소 계산	앞 또는 가까운 끝에서 순회
변경 준비	원소를 옮겨 자리 확보	새 노드 또는 양옆 노드 확인
구조 변경	값 저장	참조 변경

> 배열은 원소 이동, 연결리스트는 위치를 찾는 순회가 비용을 지배한다

Source: Morin (2013), Sections 2.1, 3.1, and 3.2.

배열의 i번째에 삽입

i부터 $n-1$까지의 원소를 오른쪽으로 한 칸씩 옮긴다



용량이 충분하면 $n-i$개를 옮기므로 전체 비용은 $\Theta(n-i+1)$이다

Source: Morin (2013), Theorem 2.1.

배열의 i번째에서 삭제

$i+1$부터 $n-1$까지의 원소를 왼쪽으로 한 칸씩 옮겨 빈자리를 메운다

삭제 위치	옮기는 수	비용
앞 $i=0$	$n-1$	$\Theta(n)$
가운데	$n-i-1$	$\Theta(n-i)$
끝 $i=n-1$	0	$\Theta(1)$

Source: Morin (2013), Theorem 2.1.

SLList의 i번째에 삽입

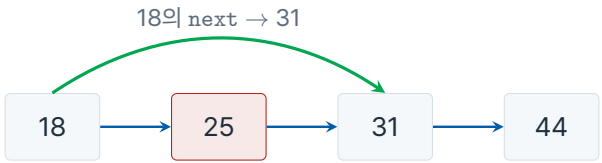
부분	하는 일	비용
위치 찾기	$i-1$번째 노드까지 순회	$\Theta(i)$
연결 변경	`next` 두 곳 변경	$\Theta(1)$
전체	찾기와 변경	$\Theta(i+1)$

> $i=0$이면 위치를 찾지 않아 앞 삽입이 $\Theta(1)$이다

Adapted from Morin (2013), Chapter 3, Exercise 3.3.

SLList에서 삭제

삭제할 노드가 아니라 그 앞 노드의 `next`를 바꾼다



Caution 삭제 위치를 인덱스로 받았다면 앞 노드를 찾는 순회 비용은 남는다

Source: Morin (2013), Section 3.1.

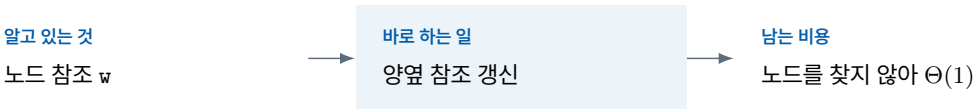
DLList는 양옆을 함께 고친다

상황	찾기	참조 변경
인덱스 i로 삽입	$\Theta(1+\min\{i, n-i\})$	$\Theta(1)$
인덱스 i로 삭제	$\Theta(1+\min\{i, n-i\})$	$\Theta(1)$
노드 참조를 이미 가짐	없음	$\Theta(1)$

Source: Morin (2013), Theorem 3.2.

노드를 이미 알고 있다면

탐색 결과나 코드가 노드 참조를 이미 가지고 있으면 위치를 다시 찾지 않는다



Source: Morin (2013), Section 3.2.1.

배열의 끝 삽입에는 두 경우가 있다

상태	하는 일	비용
용량 여유 있음	마지막 칸에 저장	$\Theta(1)$
용량 부족	더 큰 배열로 재할당·복사	$\Theta(n)$
연속한 끝 삽입	비싼 복사를 나누어 셈	상각 $\Theta(1)$

Caution 상각 $\Theta(1)$은 매번 $\Theta(1)$이라는 뜻이 아니다

Source: Morin (2013), Section 2.1.2.

연산별 비용을 비교한다

연산	ArrayStack	SLList	DLList
`get·set`	$\Theta(1)$	$\Theta(i+1)$	$\Theta(1+\min\{i, n-i\})$
`add(i,x)`	상각 $\Theta(1+n-i)$	$\Theta(i+1)$	$\Theta(1+\min\{i, n-i\})$
`remove(i)`	상각 $\Theta(1+n-i)$	$\Theta(i+1)$	$\Theta(1+\min\{i, n-i\})$
앞 삽입·삭제	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
끝 삽입	상각 $\Theta(1)$	$\Theta(1)$	$\Theta(1)$
끝 삭제	상각 $\Theta(1)$	$\Theta(n)$	$\Theta(1)$

Source: Morin (2013), Chapter 3 and Theorems 2.1, 3.1, 3.2.

Checkpoint

Insert and Remove Costs

- 배열 삽입과 연결리스트 삽입에서 반복되는 일은 각각 무엇인가
- DLList의 삽입·삭제가 $\Theta(1)$이 되려면 무엇을 이미 알아야 하는가
- 배열 끝 삽입의 최악의 경우와 상각 비용은 어떻게 다른가

Section 4 • Arrays and Linked Lists

Space and Operation Mix

시간과 공간을 함께 보고 자주 쓰는 연산에 맞춰 표현을 고른다

공간 비용도 내부 표현에서 나온다

표현	값 이외에 필요한 것	남을 수 있는 자리
ArrayStack	내부 배열과 n	사용하지 않는 용량
SLList	노드마다 `next`	없음
DLList	노드마다 `prev`, `next`	없음

> 연결리스트는 남는 용량 대신 노드마다 추가 참조를 저장한다

Source: Morin (2013), Chapters 2 and 3.

배열의 남는 용량

용량 m에 n개를 저장하면 $m - n$칸이 비지만, 용량이 $3n$ 이상이면 줄이므로 빈칸은 $2n$ 미만이다



Source: Morin (2013), Section 2.1.2.

연결리스트의 참조 비용

표현	값을 담는 자리	참조를 담는 자리
SLList 노드	값 1개	참조 1개
DLList 노드	값 1개	참조 2개

배열은 사용하지 않는 용량이 남을 수 있고, 연결리스트는 노드마다 참조를 하나 또는 두 개 더 저장한다

Source: Morin (2013), Sections 3.1 and 3.2.

호출 횟수와 연산 비용을 곱해 더한다

호출 횟수	연산별 횟수를 센다
곱하기	호출 횟수 $\times$ 비용
합 비교	표현별 전체 비용

$$\text{전체 비용} \approx \sum(\text{호출 횟수}) \times (\text{연산 비용})$$

Adapted from Morin (2013), Section 1.5.

읽기가 많은 작업

임의 접근 90회와 앞 삽입 10회를 수행한다고 하자

표현	최악 비용식	최고차항
ArrayStack	$90 \cdot 1 + 10 \cdot n$	$10n$
SLList	$90 \cdot n + 10 \cdot 1$	$90n$

> 이 작업의 최고차항은 ArrayStack이 $10n$, SLList가 $90n$이다

Adapted from Morin (2013), Theorems 2.1 and 3.1, and Chapter 3, Exercise 3.3.

앞 삽입이 많은 작업

임의 접근 10회와 앞 삽입 90회를 수행한다고 하자

표현	최악 비용식	최고차항
ArrayStack	$10 \cdot 1 + 90 \cdot n$	$90n$
SLList	$10 \cdot n + 90 \cdot 1$	$10n$

> 이 작업의 최고차항은 ArrayStack이 $90n$, SLList가 $10n$이다

Adapted from Morin (2013), Theorems 2.1 and 3.1, and Chapter 3, Exercise 3.3.

정렬된 목록도 순서대로 연결된다

값의 순서와 노드의 연결 순서가 같아도 인덱스로 바로 접근할 수는 없다



값은 오름차순이지만 77에 닿으려면 앞 노드부터 연결을 따라가야 한다

> 정렬 배열은 가운데 칸의 값을 바로 읽을 수 있지만 정렬 연결리스트는 가운데까지 연결을 따라가야 한다

Source: Miller and Ranum (2013), Implementing an Ordered List.

Checkpoint

Space and Operation Mix

- 배열과 연결리스트의 추가 공간은 각각 어디에서 생기는가
- 연산별 호출 횟수가 바뀌면 같은 n에서도 선택이 달라지는 까닭은 무엇인가
- 임의 접근과 앞 삽입이 모두 중요하다면 무엇을 먼저 측정해야 하는가

Which End Should Serve the Next Item?

Next Lecture: Stacks

- LIFO와 `push`, `pop`, `peek`
- 배열과 연결리스트로 스택 구현
- 호출 스택과 후위식 처리

오용경 yongkyung.oh@pusan.ac.kr